Veille technologique

JavaScript : le DOM, les événements et la validation de formulaires

Dans cette veille, j'ai étudié comment rendre une page web interactive avec JavaScript. Je l'ai divisée en trois parties : d'abord comment sélectionner et modifier les éléments de la page (le DOM), ensuite comment réagir aux actions de l'utilisateur (les événements), et enfin comment vérifier un formulaire en temps réel. Les exemples sont basés sur le projet de catalogue d'huile d'argan.

Partie 1 : Sélectionner et manipuler le DOM

1. C'est quoi le DOM ?

Le DOM (Document Object Model) est la représentation de ma page HTML sous forme d'arbre. Chaque balise devient un objet que JavaScript peut lire et modifier. Avant, mes pages HTML et CSS étaient statiques. Avec le DOM, je peux changer le contenu, les styles et la structure de la page sans la recharger.

2. querySelector et querySelectorAll

Ces deux méthodes utilisent des sélecteurs CSS, comme dans mon fichier CSS (`.classe`, `#id`, `input[type='email']`).

- `querySelector` renvoie seulement le **premier** élément trouvé (ou `null` s'il n'y en a pas).
- `querySelectorAll` renvoie une **NodeList** avec tous les éléments trouvés. On peut la parcourir avec `forEach`.

```
const titre = document.querySelector("h1");
const catalogue = document.querySelector("#catalogue");
const produits = document.querySelectorAll(".produit");

produits.forEach((produit) => {
  console.log(produit.textContent);
});
```

J'ai retenu qu'il faut faire attention : si `querySelector` ne trouve rien, il renvoie `null` et ça provoque une erreur quand on essaie d'utiliser l'élément.

3. Parcourir le DOM

À partir d'un élément, je peux aller vers son parent, ses enfants ou ses frères :

- `parentElement` : le parent
- `children` : les enfants
- `nextElementSibling` : le frère suivant

```
const bouton = document.querySelector(".produit button");
const carte = bouton.parentElement;
const carteSuivante = carte.nextElementSibling;
```

4. Créer et insérer un élément

Pour créer un nouvel élément, j'utilise `document.createElement`. Ensuite je le configure (texte, classe...) et je l'ajoute à la page avec `append` (à la fin), `prepend` (au début) ou `insertBefore` (avant un autre élément). Pour le texte, `textContent` met du texte simple, alors que `innerHTML` interprète le HTML.

```
const catalogue = document.querySelector("#catalogue");

const carte = document.createElement("article");
carte.classList.add("produit");

const titre = document.createElement("h3");
titre.textContent = "Huile d'argan cosmétique";

carte.append(titre);
catalogue.append(carte);
```

5. Classes et attributs

- `classList.add` / `classList.remove` : ajouter ou enlever une classe
- `classList.toggle` : l'ajoute si elle n'existe pas, l'enlève si elle existe
- `setAttribute` / `getAttribute` : modifier ou lire un attribut

```
carte.classList.add("en-promo");
carte.classList.toggle("selectionne");
carte.setAttribute("data-id", "12");
console.log(carte.getAttribute("data-id"));
```

Questions de réflexion

Pourquoi choisir querySelectorAll plutôt que plusieurs appels à querySelector ?

Parce que `querySelector` ne donne qu'un seul élément. Si je veux modifier toutes les cartes produits, je devrais faire plusieurs appels et donner un identifiant différent à chaque carte, ce qui est long. Avec `querySelectorAll`, je récupère tout d'un coup et je fais une boucle. Et si j'ajoute un produit plus tard, je n'ai pas besoin de changer mon code.

Dans quel cas faut-il privilégier createElement plutôt que du HTML brut ?

Quand le contenu vient de l'utilisateur ou d'une source externe. Avec `innerHTML`, si quelqu'un écrit du code HTML ou un script dans un champ, il peut être exécuté (faille XSS). Avec `createElement` et `textContent`, tout est traité comme du texte simple. Cela permet aussi d'ajouter directement des écouteurs d'événements sur l'élément créé.

Que se passe-t-il si on modifie la classe d'un élément avant de l'ajouter au DOM ?

Il ne se passe rien de grave : l'élément existe déjà en mémoire, donc la classe est bien ajoutée. Quand on l'insère ensuite dans la page, il s'affiche directement avec le bon style. Il n'est juste pas visible tant qu'il n'est pas dans le DOM.

Partie 2 : Les événements

1. addEventListener

Un événement, c'est une action de l'utilisateur (clic, saisie, envoi d'un formulaire...). Pour réagir, j'attache un écouteur sur un élément avec `addEventListener`. Il prend le type d'événement et une fonction qui sera exécutée quand l'événement arrive.

```
const bouton = document.querySelector("#btn-ajouter");

bouton.addEventListener("click", function (e) {
  console.log("Produit ajouté au panier");
});
```

Les événements que j'ai vus : `click`, `input`, `submit`. On peut utiliser une fonction anonyme (comme au-dessus) ou une fonction nommée si on veut pouvoir retirer l'écouteur après avec `removeEventListener`.

2. L'objet Event

La fonction reçoit automatiquement un objet `event` (souvent écrit e) qui donne des informations :

- `e.target` : l'élément exact sur lequel l'utilisateur a agi
- `e.currentTarget` : l'élément sur lequel j'ai attaché l'écouteur
- `e.preventDefault()` : annule le comportement normal du navigateur
- `e.stopPropagation()` : empêche l'événement de remonter vers les parents

```
const form = document.querySelector("#commande");

form.addEventListener("submit", (e) => {
  e.preventDefault();   // la page ne se recharge pas
  console.log("Formulaire intercepté");
});
```

3. La délégation d'événements

Quand un événement se produit sur un élément, il **remonte** aussi vers ses parents (c'est le bubbling). La délégation utilise cette remontée : au lieu de mettre un écouteur sur chaque bouton, j'en mets **un seul** sur le conteneur parent, et je vérifie avec `e.target` quel élément a été cliqué.

```
const catalogue = document.querySelector("#catalogue");

catalogue.addEventListener("click", (e) => {
  const bouton = e.target.closest(".ajouter");
  if (!bouton) return;   // on ignore les autres clics

  const carte = bouton.closest(".produit");
  console.log("Ajout de :", carte.dataset.nom);
});
```

J'utilise `closest()` parce que si le bouton contient une icône, le clic peut tomber sur l'icône et pas sur le bouton.

Questions de réflexion

Pourquoi la délégation est-elle préférable avec des éléments ajoutés dynamiquement ?

Un écouteur attaché à un élément n'existe que pour cet élément. Si je crée un nouveau produit avec JavaScript, il n'aura pas d'écouteur et le bouton ne marchera pas. Avec la délégation, l'écouteur est sur le parent qui existe déjà, donc les nouveaux boutons fonctionnent automatiquement. En plus j'ai un seul écouteur au lieu de beaucoup, donc c'est plus léger.

Dans quel cas faut-il appeler preventDefault ?

Quand je veux empêcher ce que le navigateur fait normalement. Par exemple pour un formulaire : par défaut il recharge la page à l'envoi, mais moi je veux d'abord vérifier les données. Même chose pour un lien <a> qui change de page alors que je veux faire une autre action.

Que se passe-t-il si on ne stoppe pas la propagation dans un composant imbriqué ?

L'événement remonte à tous les parents, et si un parent a un écouteur du même type, il se déclenche aussi. Par exemple, un bouton « Supprimer » dans une carte cliquable : le clic supprimerait la carte mais ouvrirait aussi la carte. Avec `stopPropagation()` je règle ça, mais il ne faut pas en abuser car ça peut casser la délégation.

Partie 3 : Validation de formulaires en temps réel

1. input, change et submit

- `input` : se déclenche à chaque modification du champ, donc à chaque lettre tapée. C'est parfait pour la validation en temps réel.
- `change` : se déclenche quand la valeur est validée, c'est-à-dire quand on quitte le champ (ou qu'on choisit une option d'un select).
- `submit` : se déclenche quand on envoie le formulaire. Je l'utilise pour la vérification finale.

```
const form = document.querySelector("#inscription");
const email = document.querySelector("#email");

email.addEventListener("input", (e) => {
  console.log(e.target.value);   // e.target = le champ modifié
});

form.addEventListener("submit", (e) => {
  e.preventDefault();
});
```

2. Les expressions régulières (regex)

Une regex est un motif qui décrit comment un texte doit être écrit. On la teste avec `regex.test(texte)`, qui renvoie `true` ou `false`. Les symboles ^ et $ veulent dire début et fin : le texte entier doit respecter le motif.

```
// Email
const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

// Téléphone marocain (06 ou 07, avec 0 ou +212)
const regexTel = /^(?:\+212|0)[67]\d{8}$/;

// Mot de passe : 8 caractères min, 1 majuscule, 1 chiffre
const regexMdp = /^(?=.*[A-Z])(?=.*\d).{8,}$/;

regexTel.test("0612345678");   // true
regexTel.test("0512345678");   // false
regexMdp.test("argan2024");    // false (pas de majuscule)
regexMdp.test("Argan2024");    // true
```

Pour comprendre et tester mes regex, j'ai utilisé le site **regex101.com**. Il explique chaque partie du motif et on peut tester des exemples valides et invalides avant de copier la regex dans le code.

3. Afficher le feedback

Pour informer l'utilisateur, je crée un de message sous le champ, et j'utilise `classList` pour changer le style (bordure rouge si erreur, verte si valide). Le texte du message change selon la validité du champ.

```
function afficherEtat(champ, valide, messageErreur) {
  let span = champ.nextElementSibling;

  if (!span || !span.classList.contains("msg")) {
    span = document.createElement("span");
    span.classList.add("msg");
```

```
    champ.after(span);
  }

  champ.classList.toggle("erreur", !valide);
  champ.classList.toggle("succes", valide);
  span.textContent = valide ? "Champ valide" : messageErreur;
}
```

4. Bloquer l'envoi si le formulaire est invalide

Dans l'événement `submit`, j'appelle `preventDefault()` d'abord, puis je vérifie tous les champs. Je n'envoie les données que si tout est valide.

```
const regles = {
  email: {
    regex: /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/,
    erreur: "Email invalide"
  },
  telephone: {
    regex: /^(?:\+212|0)[67]\d{8}$/,
    erreur: "Numéro invalide (06 ou 07)"
  },
  motdepasse: {
    regex: /^(?=.*[A-Z])(?=.*\d).{8,}$/,
    erreur: "8 caractères, 1 majuscule, 1 chiffre"
  }
};

function validerChamp(champ) {
  const regle = regles[champ.name];
  if (!regle) return true;
  const valide = regle.regex.test(champ.value.trim());
  afficherEtat(champ, valide, regle.erreur);
  return valide;
}

// validation en temps réel
form.addEventListener("input", (e) => validerChamp(e.target));

// validation à l'envoi
form.addEventListener("submit", (e) => {
  e.preventDefault();
  let toutValide = true;

  form.querySelectorAll("input[name]").forEach((champ) => {
    if (!validerChamp(champ)) toutValide = false;
  });

  if (toutValide) {
    console.log("Formulaire valide, on envoie les données");
  }
});
```

Côté HTML, j'ajoute `novalidate` sur le formulaire pour que mes messages s'affichent à la place de ceux du navigateur.

Questions de réflexion

Pourquoi valider à la saisie plutôt qu'uniquement à la soumission ?

Parce que l'utilisateur voit son erreur tout de suite et peut la corriger pendant qu'il écrit, au lieu de découvrir toutes les erreurs après avoir cliqué sur envoyer. C'est plus agréable, ça évite des envois inutiles et ça guide l'utilisateur, par exemple pour les règles du mot de passe. Mais la validation à la soumission reste nécessaire, et il faut aussi vérifier côté serveur.

Que se passe-t-il si on n'appelle pas preventDefault() sur submit alors qu'il y a des erreurs ?

Le navigateur envoie quand même le formulaire et recharge la page. Les données invalides partent vers le serveur, mes messages d'erreur disparaissent et l'utilisateur peut perdre ce qu'il a écrit. C'est pour cela qu'il faut bloquer l'envoi et ne le permettre que quand tout est valide.

Comment la délégation aide-t-elle avec des champs ajoutés dynamiquement ?

Si je mets un seul écouteur `input` sur le formulaire, il gère tous les champs, même ceux créés plus tard avec JavaScript, grâce à la remontée de l'événement. Avec `e.target` je sais quel champ a changé et avec son `name` je choisis la bonne règle. Je n'ai pas besoin de rajouter un écouteur à chaque nouveau champ.

Conclusion

Grâce à cette veille, j'ai compris que le DOM me permet de modifier la page, que les événements me permettent de réagir à l'utilisateur, et que la validation en temps réel rend les formulaires plus fiables et plus agréables. La délégation d'événements et `preventDefault` sont les deux notions que je vais utiliser le plus dans le projet du catalogue d'argan.

Sources : MDN Web Docs, javascript.info, regex101.com